

Практичне заняття №5

Автентифікація та авторизація: JWT + Guards + RBAC

Дисципліна: Високорівневі мови програмування та фреймворки

Мета

Додати систему автентифікації та авторизації до MiniShop API: створити User Entity з хешованим паролем, реалізувати реєстрацію та логін з JWT-токенами, написати JwtAuthGuard для захисту маршрутів, створити RolesGuard з декораторами @Roles() та @CurrentUser() для розмежування прав доступу. В результаті: GET-ендпоінти залишаться публічними, а POST/PATCH/DELETE будуть доступні тільки авторизованим користувачам з відповідними ролями.

Передумови

Виконане Практичне заняття №4: MiniShop API з CRUD для categories та products, DTO з class-validator, глобальний ValidationPipe, TrimPipe. Все працює через docker compose.

Що здаємо (тільки одне)

Лінк на GitHub репозиторій, в якому є:

- README.md з коротким звітом (шаблон нижче) + виводом команд
- User Entity + міграція для таблиці users
- AuthModule з реєстрацією та логіном (JWT + bcrypt)
- JwtAuthGuard + RolesGuard + декоратори @Roles(), @CurrentUser()
- Захищені ендпоінти: POST/PATCH/DELETE тільки для авторизованих
- Мінімум 4 нових коміти

Важливо: Паролі зберігаються ТІЛЬКИ у хешованому вигляді (bcrypt). Зберігання пароля у відкритому тексті = 0 балів за практичну.

Вправа 1 — Встановити пакети автентифікації

Опис

Встановити всі необхідні пакети для JWT-автентифікації: @nestjs/jwt для генерації та верифікації токенів, bcrypt для хешування паролів, та їх TypeScript-типи.

Інструкції (покроково)

1) Встановити пакети через Docker:

```
docker compose run --rm app npm install @nestjs/jwt bcrypt
docker compose run --rm app npm install -D @types/bcrypt
```

2) Додати JWT_SECRET та JWT_EXPIRES_IN до файлу .env:

```
# ... існуючі змінні ...
```

```
# JWT
```

```
JWT_SECRET=my-super-secret-key-change-in-production
```

```
JWT_EXPIRES_IN=1h
```

⚠ В production JWT_SECRET має бути довгим випадковим рядком (мінімум 32 символи). Для навчального проєкту — будь-який рядок. Але НІКОЛИ не хардкодьте секрет у коді — тільки через .env.

3) Оновити .env.example (для GitHub):

```
JWT_SECRET=change-me
```

```
JWT_EXPIRES_IN=1h
```

Очікуваний результат

- package.json містить @nestjs/jwt та bcrypt
- .env містить JWT_SECRET та JWT_EXPIRES_IN
- Застосунок запускається без помилок

Вправа 2 — Створити User Entity та міграцію

Опис

Створити модуль users із сутністю User: email (унікальний), passwordHash (хеш пароля), role (enum: user/admin), createdAt. Написати та запустити міграцію.

Інструкції (покроково)

1) Створити enum ролей — файл src/common/enums/role.enum.ts:

```
mkdir -p src/common/enums
```

```
export enum Role {
  USER = 'user',
  ADMIN = 'admin',
}
```

2) Створити файл src/users/user.entity.ts:

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  CreateDateColumn,
} from 'typeorm';
import { Role } from '../common/enums/role.enum';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  email: string;

  @Column()
  passwordHash: string;

  @Column({ length: 100, nullable: true })
  name: string;

  @Column({
    type: 'enum',
    enum: Role,
    default: Role.USER,
  })
  role: Role;

  @CreateDateColumn()
  createdAt: Date;
}
```

💡 *Поле називається passwordHash, а не password. Це важливо: ніхто, хто читає код, не подумає, що там зберігається відкритий пароль. Неймінг має значення.*

3) Створити файл src/users/users.module.ts:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { UsersService } from '../users.service';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}
```

Зверніть увагу: `exports: [UsersService]` — це ключовий рядок. `AuthModule` потребує `UsersService` для пошуку та створення користувачів. Без `exports` він буде недоступний за межами `UsersModule`.

4) Створити файл `src/users/users.service.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from '../user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly userRepo: Repository<User>,
  ) {}

  async findByEmail(email: string): Promise<User | null> {
    return this.userRepo.findOne({ where: { email } });
  }

  async create(data: Partial<User>): Promise<User> {
    const user = this.userRepo.create(data);
    return this.userRepo.save(user);
  }
}
```

5) Зареєструвати User Entity в `app.module.ts` та додати `UsersModule` до `imports`:
Додати `User` до масиву `entities` в `TypeOrmModule.forRoot` та `UsersModule` до `imports AppModule`.

6) Згенерувати міграцію та перезапустити:

```
docker compose run --rm app npx ts-node -r tsconfig-paths/register \
  ./node_modules/typeorm/cli.js migration:generate \
  src/migrations/CreateUsers -d src/data-source.ts

docker compose up --build -d
```

```
docker compose logs -f app
```

7) Перевірити, що таблиця створена:

```
docker compose exec postgres psql -U nestuser -d nestdb -c "\d users"
```

Очікується: таблиця users з колонками id, email, passwordHash, name, role, createdAt.

Очікуваний результат

- Таблиця users створена міграцією
- UsersModule зареєстрований та експортує UsersService
- Застосунок запускається без помилок

Вправа 3 — Створити Auth DTO

Опис

Створити RegisterDto та LoginDto з валідацією. Окремі DTO для реєстрації та логіну — бо їх поля відрізняються (реєстрація потребує name, логін — ні).

Інструкції (покроково)

1) Створити файл src/auth/dto/register.dto.ts:

```
mkdir -p src/auth/dto
```

```
import {
  IsEmail,
  IsString,
  MinLength,
  MaxLength,
  IsOptional,
} from 'class-validator';

export class RegisterDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  @MaxLength(128)
  password: string;

  @IsOptional()
  @IsString()
```

```
@MaxLength(100)
name?: string;
}
```

💡 *MaxLength(128) для пароля — це не випадкове число. bcrypt має обмеження: він обробляє тільки перші 72 байти вхідних даних. Пароль довший за 128 символів — підозрілий і, ймовірно, атака.*

2) Створити файл src/auth/dto/login.dto.ts:

```
import { IsEmail, IsString } from 'class-validator';

export class LoginDto {
  @IsEmail()
  email: string;

  @IsString()
  password: string;
}
```

Очікуваний результат

- src/auth/dto/register.dto.ts — з @IsEmail, @MinLength(8)
- src/auth/dto/login.dto.ts — мінімальна валідація для логіну

Вправа 4 — Створити AuthModule та AuthService

Опис

Створити AuthService з двома ключовими методами: register (хешування + збереження) та login (пошук + порівняння + генерація JWT). Створити AuthModule, який імпортує UsersModule та JwtModule.

Інструкції (покроково)

1) Створити файл src/auth/auth.service.ts:

```
import {
  Injectable,
  ConflictException,
  UnauthorizedException,
} from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import { UsersService } from '../users/users.service';
import { RegisterDto } from '../dto/register.dto';
```

```

import { LoginDto } from '../dto/login.dto';

@Injectable()
export class AuthService {
  constructor(
    private readonly usersService: UsersService,
    private readonly jwtService: JwtService,
  ) {}

  async register(dto: RegisterDto) {
    // 1. Перевірити унікальність email
    const existing = await this.usersService.findByEmail(
      dto.email,
    );
    if (existing) {
      throw new ConflictException(
        'User with this email already exists',
      );
    }

    // 2. Хешувати пароль (rounds = 10)
    const passwordHash = await bcrypt.hash(
      dto.password,
      10,
    );

    // 3. Зберегти користувача
    const user = await this.usersService.create({
      email: dto.email,
      passwordHash,
      name: dto.name,
    });

    // 4. Повернути без хешу пароля
    const { passwordHash: _, ...result } = user;
    return result;
  }

  async login(dto: LoginDto) {
    // 1. Знайти користувача за email
    const user = await this.usersService.findByEmail(
      dto.email,
    );
    if (!user) {
      throw new UnauthorizedException(
        'Invalid credentials',
      );
    }
  }
}

```

```

    );
  }

  // 2. Порівняти пароль
  const isMatch = await bcrypt.compare(
    dto.password,
    user.passwordHash,
  );
  if (!isMatch) {
    throw new UnauthorizedException(
      'Invalid credentials',
    );
  }

  // 3. Згенерувати JWT
  const payload = {
    sub: user.id,
    email: user.email,
    role: user.role,
  };

  return {
    accessToken: this.jwtService.sign(payload),
  };
}
}

```

⚠️ Однакове повідомлення 'Invalid credentials' для обох випадків (користувач не знайдений і неправильний пароль) — це свідомий захист від user enumeration. Ми не даємо підказок зломиснику, чи існує такий email.

2) Створити файл src/auth/auth.module.ts:

```

import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { UsersModule } from '../users/users.module';
import { AuthService } from './auth.service';
import { AuthController } from './auth.controller';

@Module({
  imports: [
    UsersModule,
    JwtModule.registerAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({

```



```

    secret: config.get<string>('JWT_SECRET'),
    signOptions: {
      expiresIn: config.get<string>(
        'JWT_EXPIRES_IN',
        '1h',
      ),
    },
  },
  providers: [AuthService],
  exports: [JwtModule],
})
export class AuthModule {}

```

Пояснення: `JwtModule.registerAsync` дозволяє конфігурувати JWT з `.env` через `ConfigService`. `exports: [JwtModule]` — `Guard`'и в інших модулях потребують `JwtService` для верифікації токенів.

Очікуваний результат

- `AuthService` реалізує `register` (`bcrypt.hash + save`) та `login` (`compare + jwt.sign`)
- `AuthModule` імпортує `UsersModule` та `JwtModule`

Вправа 5 — Створити `AuthController`

Опис

Створити контролер з двома ендпоінтами: `POST /auth/register` та `POST /auth/login`. Контролер тонкий — він лише делегує роботу `AuthService`.

Інструкції (покроково)

1) Створити файл `src/auth/auth.controller.ts`:

```

import {
  Controller,
  Post,
  Body,
  HttpStatusCode,
  HttpStatus,
} from '@nestjs/common';
import { AuthService } from '../auth.service';
import { RegisterDto } from '../dto/register.dto';

```

```
import { LoginDto } from '../dto/login.dto';

@Controller('auth')
export class AuthController {
  constructor(
    private readonly authService: AuthService,
  ) {}

  @Post('register')
  register(@Body() dto: RegisterDto) {
    return this.authService.register(dto);
  }

  @Post('login')
  @HttpCode(HttpStatus.OK)
  login(@Body() dto: LoginDto) {
    return this.authService.login(dto);
  }
}
```

💡 *@HttpCode(HttpStatus.OK) на login потрібен, бо за замовчуванням @Post() повертає 201 Created. Але логін не створює ресурс — він повертає токен. Тому коректний статус — 200 OK.*

2) Зареєструвати AuthModule в app.module.ts — додати до imports масиву.

3) Перезапустити та перевірити маршрути:

```
docker compose restart app
docker compose logs -f app
```

В логах мають з'явитись нові маршрути:

```
[RoutesResolver] AuthController {/auth}:
[RouterExplorer] Mapped {/auth/register, POST}
[RouterExplorer] Mapped {/auth/login, POST}
```

4) Протестувати реєстрацію та логін:

```
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@test.com", "password": "password123", "name": "Admin"}'
```

Очікується: 201 Created, JSON з id, email, name, role ("user"), createdAt. Без passwordHash!

```
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
```

```
-d '{"email": "admin@test.com", "password": "password123"}'
```

Очікується: 200 OK, JSON з accessToken. Збережіть цей токен — він знадобиться для наступних вправ!

Очікуваний результат

- POST /auth/register — створює користувача, повертає без пароля
- POST /auth/login — повертає { accessToken }
- Повторна реєстрація з тим же email — 409 Conflict

Вправа 6 — Створити JwtAuthGuard

Опис

Створити Guard, який перевіряє JWT-токен із заголовка Authorization: Bearer <token>. Guard витягує токен, верифікує його і прикріплює payload до request.user. Якщо токена немає або він невалідний — повертає 401 Unauthorized.

Інструкції (покроково)

1) Створити файл src/common/guards/jwt-auth.guard.ts:

```
mkdir -p src/common/guards
```

```
import {
  Injectable,
  CanActivate,
  ExecutionContext,
  UnauthorizedException,
} from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { Request } from 'express';

@Injectable()
export class JwtAuthGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context
      .switchToHttp()
      .getRequest<Request>();

    const token = this.extractToken(request);
    if (!token) {
```

```

        throw new UnauthorizedException(
            'Missing authorization token',
        );
    }

    try {
        const payload = this.jwtService.verify(token);
        request['user'] = payload;
    } catch {
        throw new UnauthorizedException(
            'Invalid or expired token',
        );
    }

    return true;
}

private extractToken(
    request: Request,
): string | undefined {
    const [type, token] =
        request.headers.authorization?.split(' ') ?? [];
    return type === 'Bearer' ? token : undefined;
}
}

```

Пояснення роботи Guard

Крок	Що відбувається
extractToken	Читає заголовок Authorization, перевіряє формат "Bearer <token>", повертає сам токен
jwtService.verify	Перевіряє підпис і термін дії. Якщо невалідно — кидає помилку
request['user']	Прикріплює payload (sub, email, role) до request. Тепер контролер може його прочитати
return true	Пропускає запит далі по конвеєру

Очікуваний результат

- Файл src/common/guards/jwt-auth.guard.ts створений
- Guard реалізує CanActivate, використовує JwtService через DI

Вправа 7 — Створити декоратори @CurrentUser() та @Roles()

Опис

Створити два кастомних декоратори: @CurrentUser() для типобезпечного доступу до поточного користувача та @Roles() для позначення, які ролі мають доступ до ендпоінта.

Інструкції (покроково)

1) Створити файл src/common/decorators/current-user.decorator.ts:

```
mkdir -p src/common/decorators
```

```
import {
  createParamDecorator,
  ExecutionContext,
} from '@nestjs/common';

export const CurrentUser = createParamDecorator(
  (data: string | undefined, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    const user = request.user;
    return data ? user?.[data] : user;
  },
);
```

Як використовувати: @CurrentUser() user — повний об'єкт. @CurrentUser('id') userId: number — тільки id. Як @Body() та @Body('name') — та сама логіка.

2) Створити файл src/common/decorators/roles.decorator.ts:

```
import { SetMetadata } from '@nestjs/common';
import { Role } from '../../enums/role.enum';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: Role[]) =>
  SetMetadata(ROLES_KEY, roles);
```

 *SetMetadata прикріплює дані до методу або класу через Reflect API. Самі по собі ці metadata нічого не роблять — їх читає RolesGuard у наступній вправі.*

Очікуваний результат

- `@CurrentUser()` повертає типізований об'єкт користувача
- `@Roles(Role.ADMIN)` позначає метод як доступний тільки для адмінів

Вправа 8 — Створити RolesGuard

Опис

Створити Guard, який читає metadata з `@Roles()` через `Reflector` і перевіряє, чи роль поточного користувача є в списку дозволених. Якщо `@Roles()` не встановлений — пропускає запит (доступ для всіх авторизованих).

Інструкції (покроково)

1) Створити файл `src/common/guards/roles.guard.ts`:

```
import {
  Injectable,
  CanActivate,
  ExecutionContext,
  ForbiddenException,
} from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { Role } from '../enums/role.enum';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector
      .getAllAndOverride<Role[]>(ROLES_KEY, [
        context.getHandler(),
        context.getClass(),
      ]);

    // Якщо @Roles() не встановлений — доступ відкритий
    if (!requiredRoles || requiredRoles.length === 0) {
      return true;
    }

    const { user } = context
      .switchToHttp()
      .getRequest();
```

```

        if (!requiredRoles.includes(user.role)) {
            throw new ForbiddenException(
                'Insufficient permissions',
            );
        }

        return true;
    }
}

```

⚠️ Порядок Guards у @UseGuards() має значення! Завжди: @UseGuards(JwtAuthGuard, RolesGuard). Спочатку JwtAuthGuard кладе user на request, потім RolesGuard читає user.role. Якщо навпаки — RolesGuard не знайде user і впаде з помилкою.

Очікуваний результат

- RolesGuard читає @Roles() metadata через Reflector
- Повертає 403 Forbidden якщо роль не збігається
- Пропускає запит якщо @Roles() не встановлений

Вправа 9 — Захистити ендпоінти Products та Categories

Опис

Оновити контролери: GET-ендпоінти залишаються публічними (каталог доступний всім). POST, PATCH, DELETE — тільки для авторизованих з роллю ADMIN. Це відповідає бізнес-логіці: покупці переглядають каталог, адміни керують товарами.

Інструкції (покроково)

1) Оновити src/products/products.controller.ts — додати імпорти та Guards:

```

import {
    Controller, Get, Post, Patch, Delete,
    Param, Body, ParseIntPipe, UseGuards,
} from '@nestjs/common';
import { ProductsService } from '../products.service';
import { CreateProductDto } from '../dto/create-product.dto';
import { UpdateProductDto } from '../dto/update-product.dto';
import { JwtAuthGuard }
    from '../common/guards/jwt-auth.guard';

```

```

import { RolesGuard }
  from '../common/guards/roles.guard';
import { Roles }
  from '../common/decorators/roles.decorator';
import { Role } from '../common/enums/role.enum';

@Controller('api/products')
export class ProductsController {
  constructor(
    private readonly productService: ProductService,
  ) {}

  // Публічні ендпоінти — без Guard
  @Get()
  findAll() {
    return this.productService.findAll();
  }

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.productService.findOne(id);
  }

  // Захищені ендпоінти — тільки ADMIN
  @Post()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(Role.ADMIN)
  create(@Body() dto: CreateProductDto) {
    return this.productService.create(dto);
  }

  @Patch(':id')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(Role.ADMIN)
  update(
    @Param('id', ParseIntPipe) id: number,
    @Body() dto: UpdateProductDto,
  ) {
    return this.productService.update(id, dto);
  }

  @Delete(':id')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(Role.ADMIN)
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.productService.remove(id);
  }
}

```



```
}  
}
```

2) Аналогічно оновити `src/categories/categories.controller.ts` — додати ті самі Guards та `@Roles(Role.ADMIN)` на POST, PATCH, DELETE. GET залишити публічними.

3) Зареєструвати `JwtAuthGuard` як provider в `ProductsModule` та `CategoriesModule`:

Guard використовує `JwtService` через DI, тому модуль повинен імпортувати `AuthModule` (який експортує `JwtModule`):

```
// src/products/products.module.ts  
import { Module } from '@nestjs/common';  
import { TypeOrmModule } from '@nestjs/typeorm';  
import { Product } from '../product.entity';  
import { ProductsService } from './products.service';  
import { ProductsController } from './products.controller';  
import { AuthModule } from '../../auth/auth.module';  
  
@Module({  
  imports: [  
    TypeOrmModule.forFeature([Product]),  
    AuthModule,  
  ],  
  controllers: [ProductsController],  
  providers: [ProductsService],  
})  
export class ProductsModule {}
```

Аналогічно для `CategoriesModule`.

4) Створити адміна напямучу в базі (для тестування):

Зареєструйте звичайного користувача через API, потім оновіть його роль:

```
docker compose exec postgres psql -U nestuser -d nestdb \  
-c "UPDATE users SET role = 'admin' WHERE email = 'admin@test.com';"
```

5) Перезапустити:

```
docker compose restart app
```

Очікуваний результат

- GET `/api/products`, GET `/api/categories` — працюють без токена
- POST/PATCH/DELETE без токена — 401 Unauthorized
- POST/PATCH/DELETE з токеном user — 403 Forbidden

- POST/PATCH/DELETE з токеном admin — працюють

Вправа 10 — Тестування всієї системи через curl

Опис

Повне тестування auth flow: реєстрація, логін, захищені запити з токеном, перевірка ролей, перевірка помилок. Це ваш ручний інтеграційний тест.

Auth flow

1. Реєстрація нового користувача:

```
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email": "user@test.com", "password": "securepass123", "name": "Test User"}'
```

Очікується: 201 Created, JSON без passwordHash.

2. Повторна реєстрація (дубль email):

```
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email": "user@test.com", "password": "otherpass123"}'
```

Очікується: 409 Conflict.

3. Логін:

```
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "user@test.com", "password": "securepass123"}'
```

Очікується: 200 OK, { "accessToken": "eyJhbGciOi..." }.

Збережіть токен у змінну (Linux/Mac):

```
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@test.com", "password": "password123"}' | grep -o
'"accessToken": "[^"]*"') | cut -d'"' -f4)
echo $TOKEN
```

4. Невірний пароль:

```
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "user@test.com", "password": "wrongpassword"}'
```

Очікується: 401 Unauthorized, "Invalid credentials".

Тести авторизації

5. GET без токена (публічний):

```
curl http://localhost:3000/api/products
```

Очікується: 200 OK, масив продуктів.

6. POST без токена:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Hacked Product", "price": 1}'
```

Очікується: 401 Unauthorized.

7. POST з токеном USER (не admin):

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <USER_TOKEN>" \
  -d '{"name": "Blocked Product", "price": 99}'
```

Очікується: 403 Forbidden, "Insufficient permissions".

8. POST з токеном ADMIN:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <ADMIN_TOKEN>" \
  -d '{"name": "MacBook Pro", "price": 2499.99, "stock": 10}'
```

Очікується: 201 Created.

9. DELETE з токеном ADMIN:

```
curl -X DELETE http://localhost:3000/api/products/1 \
  -H "Authorization: Bearer <ADMIN_TOKEN>"
```

Очікується: 200 OK.

💡 Замініть <USER_TOKEN> та <ADMIN_TOKEN> на реальні токени з логіну. Або використовуйте змінну \$TOKEN, якщо зберегли її раніше.

Вправа 11 — Оновити README та зробити коміти

Інструкції (покроково)

1) Переконайтесь, що .env додано до .gitignore.

2) Зробити коміти (мінімум 4):

```
git add package.json package-lock.json .env.example
git commit -m "add jwt and bcrypt packages, jwt config"

git add src/common/enums/ src/users/
git commit -m "add User entity, UsersModule, migration"

git add src/auth/
git commit -m "add AuthModule with register, login, JWT"

git add src/common/guards/ src/common/decorators/
git add src/products/ src/categories/
git commit -m "add JwtAuthGuard, RolesGuard, protect endpoints"

git add README.md
git commit -m "update readme for practical 5"

git push
```

3) Оновити README.md за шаблоном:

```
## Student
- Name: <ПІБ>
- Group: <Група>

## Практичне заняття №5 – JWT Authentication + Guards + RBAC

### Структура репозиторію
...

.
├─ src/
│   ├─ auth/
│   │   ├─ dto/
│   │   │   ├─ register.dto.ts
│   │   │   └─ login.dto.ts
│   │   └─ auth.module.ts
│   │       ├─ auth.service.ts
│   │       └─ auth.controller.ts
│   └─ users/
│       ├─ user.entity.ts
│       ├─ users.module.ts
│       └─ users.service.ts
│   └─ common/
│       └─ enums/
```

```

|   |   |   └─ role.enum.ts
|   |   └─ guards/
|   |       └─ jwt-auth.guard.ts
|   |       └─ roles.guard.ts
|   |   └─ decorators/
|   |       └─ current-user.decorator.ts
|   |       └─ roles.decorator.ts
|   └─ pipes/
|       └─ trim.pipe.ts
└─ categories/ ...
└─ products/ ...
└─ migrations/
└─ data-source.ts
└─ main.ts
└─ app.module.ts
└─ Dockerfile
└─ docker-compose.yml
└─ README.md
...

```

Запуск проекту

```

```bash
cp .env.example .env
docker compose up --build
...

```

### ### API Endpoints

Method	URL	Auth	Role
POST	/auth/register	-	-
POST	/auth/login	-	-
GET	/api/categories	-	-
POST	/api/categories	JWT	admin
GET	/api/products	-	-
POST	/api/products	JWT	admin
PATCH	/api/products/:id	JWT	admin
DELETE	/api/products/:id	JWT	admin

### ### Тест реєстрації

```

```text
<вивід curl POST /auth/register>
...

```

Тест логіну

```

```text
<вивід curl POST /auth/login>

```

```
```

### Тест 401 — запит без токена
```text
<вивід curl POST /api/products без Authorization>
```

### Тест 403 — запит з роллю user
```text
<вивід curl POST /api/products з токеном user>
```

### Тест успішного створення від admin
```text
<вивід curl POST /api/products з токеном admin>
```
```

Troubleshooting — типові проблеми

1) "Cannot read properties of undefined (reading 'role')" в RolesGuard

Симптоми: 500 Internal Server Error при зверненні до захищеного ендпоінта.

Що зробити:

- Перевірити порядок Guards: `@UseGuards(JwtAuthGuard, RolesGuard)` — `JwtAuthGuard` ПЕРШИЙ
- Перевірити, що `JwtAuthGuard` прикріплює `user` до `request`: `request['user'] = payload`
- Перевірити, що `payload` JWT містить поле `role`

2) "Nest can't resolve dependencies of the JwtAuthGuard" або подібне

Симптоми: помилка при запуску застосунку.

Що зробити:

- Перевірити, що модуль, де використовується `Guard`, імпортує `AuthModule`
- Перевірити, що `AuthModule` має `exports: [JwtModule]`
- Перезібрати: `docker compose up --build`

3) "invalid signature" при verify

Симптоми: 401 при зверненні з токеном, який раніше працював.

Що зробити:

- Перевірити, що JWT_SECRET однаковий в .env при генерації та верифікації
- Якщо змінили JWT_SECRET — всі старі токени стають невалідними (це нормально)
- Залогінитись заново для отримання нового токена

4) Токен працює, але role завжди 'user' — не можу створити admin

Симптоми: 403 навіть після UPDATE в базі.

Що зробити:

- Роль зберігається в JWT при логіні. Після зміни ролі в базі потрібно ЗАЛОГІНІТИСЬ ЗАНОВО
- Перевірити: скопіюйте токен, вставте на jwt.io — чи правильний role у payload?
- Команда для перевірки ролі в базі: `docker compose exec postgres psql -U nestuser -d nestdb -c "SELECT id, email, role FROM users;"`

5) "relation 'users' does not exist"

Симптоми: помилка при реєстрації або логіні.

Що зробити:

- Перевірити, що міграція для таблиці users створена та додана до масиву migrations в app.module.ts
- Перевірити, що migrationsRun: true в TypeOrmModule конфігурації
- Перезапустити: `docker compose down && docker compose up --build`

6) "Module '...bcrypt' has no exported member"

Симптоми: помилка компіляції TypeScript.

Що зробити:

- Перевірити, що @types/bcrypt встановлений: `npm ls @types/bcrypt`
- Використовувати `import * as bcrypt from 'bcrypt'` (не `import bcrypt from 'bcrypt'`)